CS/CE 224/272 - Object Oriented Programming and Design Methodology

Nadia Nasir

Habib University, Fall 2025

The `this` Pointer

It’s hidden from you (kind of). You can use it explicitly if you want to though.

https://www.learncpp.com/cpp-tutorial/the-hidden-this-pointer-and-member-function-chaining/

How does the compiler interprets member function calls?

- We have already seen this before actually.

- Let’s take the `Complex` numbers example from your lab.

How does the compiler interprets member function calls?

- Recall that you had a member function (or method) call like so.
  - `c1` and `c2` were two `Complex` type objects in your program.

```
c1.add(c2);
```

- Compiler interprets the above as follows.

```
Complex::add(&c1, c2);
```

- c1 is now being passed *(implicitly)* by address to the function.

How does the compiler interprets member function calls?

- Note, however, that your member function looks like as follows.

- It only has a single argument.

- Whereas the compiler is now passing **two arguments**.

```
Complex Complex::add(Complex c2)
{
    Complex c3;
    c3.m_real = m_real + c2.m_real;
    c3.m_imag = m_imag + c2.m_imag;

    return c3;
}


// Compiler's interpretation
add(&c1, c2);
```

How does the compiler interprets member function calls?

- The compiler **also** updates your member function as shown.

```
c1.add(c2);
```

Compiler's interpretation

```
add(&c1, c2);
```

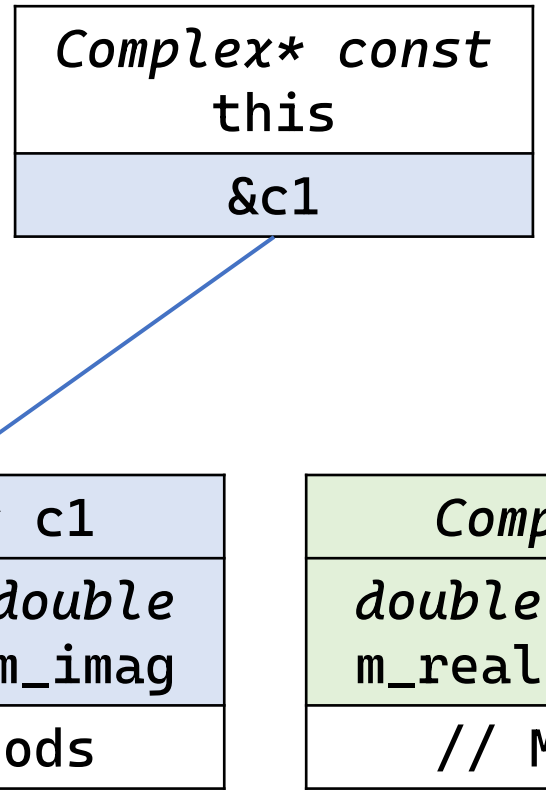
```
Complex Complex::add(Complex* const this, Complex c2)
{
    Complex c3;

    c3.m_real = this->m_real + c2.m_real;

    c3.m_imag = this->m_imag + c2.m_imag;

    return c3;
}
```

```
Complex Complex::add(Complex& c2)
{
    Complex c3;

    // c3.m_real = this->m_real + c2.m_real;
    // c3.m_imag = this->m_imag + c2.m_imag;

    this = nullptr;

    this = &c2;

    return c3;
}
```

```
c1.add(c2);
```

Compiler's interpretation

```
add(&c1, c2);
```



Both of these objects were created in `main()`.
`c3` *would be created, and be local to* `add()`.

```
Complex Complex::add(Complex& c2)
{
    Complex c3;

    // c3.m_real = this->m_real + c2.m_real;
    // c3.m_imag = this->m_imag + c2.m_imag;

    this = nullptr;

    this = &c2;

    return c3;
}
```

```
c1.add(c2);
```

Compiler's interpretation

```
add(&c1, c2);
```

Neither of these is allowed because `this` *is a* `const` ***pointer to object****; its value cannot be changed.*

```
.\rough3.cpp: In member function 'Complex Complex::add(Complex)':
.\rough3.cpp:26:12: error: lvalue required as left operand of assignment
     this = nullptr;
            ^~~~~~~
.\rough3.cpp:28:13: error: lvalue required as left operand of assignment
     this = &c2;
             ^~
```

Note

We haven't talked about lvalues and rvalues in this course.

In simple terms, you can think of lvalue as a modifiable variable.

The hidden **“this”** pointer

- Note that this **`this`** pointer is a **`const`** **pointer**.

```
Complex Complex::add(Complex* const this, Complex c2)
```

- This means that you cannot change **`this`**’s value.
  - It can only point to the implicit object.
  - You cannot change its value (not even to **`nullptr`**).
  - You can change the attributes of the entity to which it is pointing at (which is the implicit object).

The hidden **“this”** pointer

- **this** pointer will always be the leftmost parameter.

```
Complex Complex::add(Complex* const this, Complex c2)
```

In learncpp, they've created `class Simple`*, and instantiated an object of this class,* `Simple simple`*.*

Putting it all together:

1. When we call `simple.setID(2)`, the compiler actually calls `Simple::setID(&simple, 2)`, and `simple` is passed by address to the function.
2. The function has a hidden parameter named `this` which receives the address of `simple`.
3. Member variables inside setID() are prefixed with `this->`, which points to `simple`. So when the compiler evaluates `this->m_id`, it's actually resolving to `simple.m_id`.

The good news is that all of this happens automatically, and it doesn't really matter whether you remember how it works or not. All you need to remember is that all non-static member functions have a `this` pointer that refers to the object the function was called on.

Key insight

All non-static member functions have a `this` const pointer that holds the address of the implicit object.

Note

The members we have seen thus far are non-static members.
We haven't talked about static members yet.

```cpp
#include<iostream>

class A
{
private:
    int m_num1;

public:
    A() : m_num1{ 0 } {}

    int getNum1() const
    {
        this->m_num1 = 5;
        return this->m_num1;    // same as "return m_num1;"
    }

};

int main()
{
    A ob;
    ob.getNum1();

    return 0;
}
```

Self Study

Is this program valid?

Some use-cases of `this`

First up, is **Method Chaining**

```
class Calc
{
private:
    int m_value;

public:
    Calc() : m_value{ 0 } {}

    void add(int value) { m_value += value; }
    void sub(int value) { m_value -= value; }
    void mult(int value) { m_value *= value; }

    int getValue() const { return m_value; }
};
```

```
int main()
{
    Calc calc{};
    calc.add(5); // returns void
    calc.sub(3); // returns void
    calc.mult(4); // returns void

    std::cout << calc.getValue() << '\n';

    return 0;
}
```

The following will work.

What will be the output?

8

This is somewhat of a hassle; three lines.
How could we do something like so?

```
calc.add(5).sub(3).mult(4);
```

We can’t have the functions as `void`, otherwise chaining won’t work.
Something has to be returned.

We can **return by reference**.

```
class Calc
{
private:
    int m_value{};

public:
    Calc() : m_value{ 0 } {}

    Calc& add(int value) { m_value += value; }
    Calc& sub(int value) { m_value -= value; }
    Calc& mult(int value) { m_value *= value; }

    int getValue() const { return m_value; }
};
```

We can't have the functions as `void`, otherwise chaining won't work.
Something has to be returned.

We can **return by reference**.

```
class Calc
{
private:
    int m_value{};

public:
    Calc() : m_value{ 0 } {}

    Calc& add(int value) { m_value += value; return *this; }
    Calc& sub(int value) { m_value -= value; return *this; }
    Calc& mult(int value) { m_value *= value; return *this; }

    int getValue() const { return m_value; }
};
```

You go to `Calc::add()`. The value of the `calc` object changes from 0 to 5.
What gets returned?

The object, `calc`, itself.

```
calc.add(5).sub(3).mult(4);
```

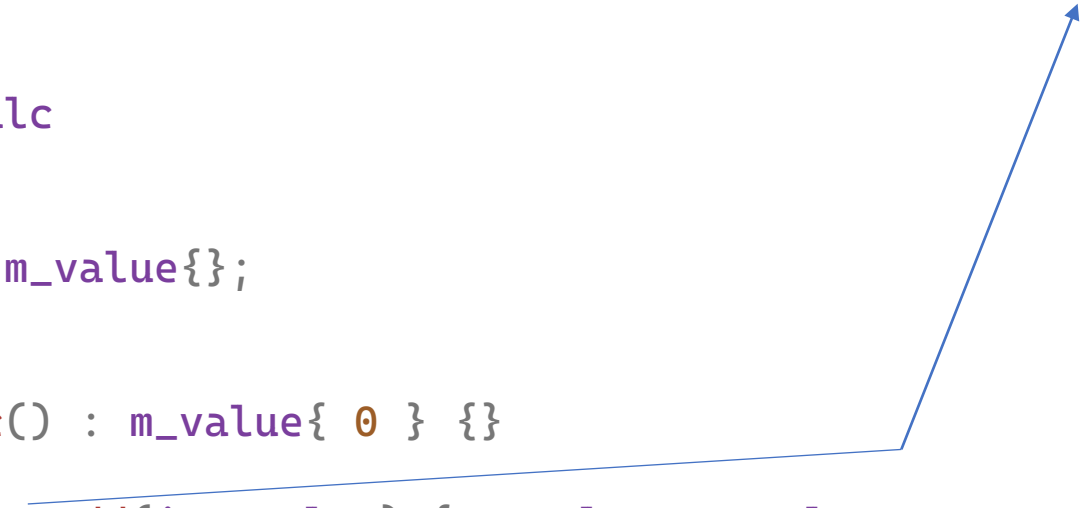
```
class Calc
{
private:
    int m_value{};

public:
    Calc() : m_value{ 0 } {}

    Calc& add(int value) { m_value += value; return *this; }
    Calc& sub(int value) { m_value -= value; return *this; }
    Calc& mult(int value) { m_value *= value; return *this; }

    int getValue() const { return m_value; }
};
```

You go to `Calc::add()`. The value of the `calc` object changes from 0 to 5.
What gets returned?

The object, `calc`, itself.

```
calc.add(5).sub(3).mult(4);
```

`calc`



```
class Calc
{
private:
    int m_value{};

public:
    Calc() : m_value{ 0 } {}

    Calc& add(int value) { m_value += value; return *this; }
    Calc& sub(int value) { m_value -= value; return *this; }
    Calc& mult(int value) { m_value *= value; return *this; }

    int getValue() const { return m_value; }
};
```

Because `calc` was returned, now we can continue our evaluation.

3 will be subtracted from the current value (which is 5) to give 2.
`calc` will be returned again.

```
calc.sub(3).mult(4);
```

```
class Calc
{
private:
    int m_value{};

public:
    Calc() : m_value{ 0 } {}

    Calc& add(int value) { m_value += value; return *this; }
    Calc& sub(int value) { m_value -= value; return *this; }
    Calc& mult(int value) { m_value *= value; return *this; }

    int getValue() const { return m_value; }
};
```

Finally 4 gets multiplied to the current value (2) to give 8.

```
calc.mult(4);
```

```
class Calc
{
private:
    int m_value{};

public:
    Calc() : m_value{ 0 } {}

    Calc& add(int value) { m_value += value; return *this; }
    Calc& sub(int value) { m_value -= value; return *this; }
    Calc& mult(int value) { m_value *= value; return *this; }

    int getValue() const { return m_value; }
};
```

`calc` is returned again, but nothing happens since there are no further operations left.

```
calc;
```

```
class Calc
{
private:
    int m_value{};

public:
    Calc() : m_value{ 0 } {}

    Calc& add(int value) { m_value += value; return *this; }
    Calc& sub(int value) { m_value -= value; return *this; }
    Calc& mult(int value) { m_value *= value; return *this; }

    int getValue() const { return m_value; }
};
```

When you do `calc.getValue()`, you see the updated value of `calc's` attribute.

We will see these concepts of **method chaining** and **return by reference** again in **operator overloading**.

```
class Calc
{
private:
    int m_value{};

public:
    Calc() : m_value{ 0 } {}

    Calc& add(int value) { m_value += value; return *this; }
    Calc& sub(int value) { m_value -= value; return *this; }
    Calc& mult(int value) { m_value *= value; return *this; }

    int getValue() const { return m_value; }
};
```

```
int main()
{
    Calc calc{};

    // method chaining
    calc.add(5).sub(3).mult(4);

    std::cout << calc.getValue() << '\n';

    return 0;
}
```

What happens if your methods **return by address**?

```
class Calc
{
private:
    int m_value{};

public:
    Calc() : m_value{ 0 } {}

    Calc* add(int value) { m_value += value;
    Calc* sub(int value) { m_value -= value;
    Calc* mult(int value) { m_value *= value;

    int getValue() const { return m_value; }
};
```

```
int main()
{
    Calc calc{};
                                                // method chaining

    std::cout << calc.getValue() << '\n';

    return 0;
}
```

}

What happens if your methods **return by address**?

```
class Calc
{
private:
    int m_value{};

public:
    Calc() : m_value{ 0 } {}

    Calc* add(int value) { m_value += value; return this; }
    Calc* sub(int value) { m_value -= value; return this; }
    Calc* mult(int value) { m_value *= value; return this; }

    int getValue() const { return m_value; }
};
```

```
int main()
{
    Calc calc{};
    [redacted] // method chaining

    std::cout << calc.getValue() << '\n';

    return 0;
}
```

What happens if your methods **return by address**?

```
class Calc
{
private:
    int m_value{};

public:
    Calc() : m_value{ 0 } {}

    Calc* add(int value) { m_value += value; return this; }
    Calc* sub(int value) { m_value -= value; return this; }
    Calc* mult(int value) { m_value *= value; return this; }

    int getValue() const { return m_value; }
};
```

```
int main()
{
    Calc calc{};
    calc.add(5)->sub(3)->mult(4); // method chaining

    std::cout << calc.getValue() << '\n';

    return 0;
}
```

What happens if your methods **return by value**?

What will be the output?
5

```
class Calc
{
private:
    int m_value{};

public:
    Calc() : m_value{ 0 } {}

    Calc add(int value) { m_value += value; return *this; }
    Calc sub(int value) { m_value -= value; return *this; }
    Calc mult(int value) { m_value *= value; return *this; }

    int getValue() const { return m_value; }
};
```

```
int main()
{
    Calc calc{};
    calc.add(5).sub(3).mult(4); // method chaining

    std::cout << calc.getValue() << '\n';

    return 0;
}
```

Another use-case is to guard against self-referencing

- Read the following slides as self-study.

```cpp
class A
{
private:
    int m_num1;

public:
    A() : m_num1{ -1 } {}

    void printAddress(A* addr)
    {
        if(addr == this)
        {
            std::cout << "Value of \"this\" = "
                      << addr << std::endl;
        }
        else
        {
            std::cout << "Some other object's address = "
                      << addr << std::endl;
        }
    }
};
```

Self Study

Guarding against self-referencing. Good idea to probably draw the things that are happening in `main()`.

```cpp
int main()
{
    A ob1;
    A ob2;

    A* ptr1{ &ob1 };

    std::cout << "Print 1\n";
    ptr1->printAddress(ptr1);

    A* ptr2{ &ob1 };
    std::cout << "Print 2\n";
    ptr2->printAddress(ptr1);

    std::cout << "Print 3\n";
    ob1.printAddress(&ob1);

    std::cout << "Print 4\n";
    ob1.printAddress(&ob2);

    std::cout << "Print 5\n";
    ob2.printAddress(&ob2);

    return 0;
}
```